

如何在 Cloud Run 部署 Gemini 技術支援的即時通訊應用程式

來源：<https://codelabs.developers.google.com/codelabs/how-to-deploy-gemini-powered-chat-app-cloud-run?hl=zh-tw>

步驟數：11

瞭解如何使用 express.js、htmx 和 tailwindCSS，在 Cloud Run 中部署採用 Gemini 的對話。

目錄

1. [1. 簡介](#)
2. [2. 設定和需求](#)
3. [3. 啟用 API 並設定環境變數](#)
4. [4. 建立及設定 Firebase 專案](#)
5. [5. 建立服務帳戶](#)
6. [6. 建立 Cloud Run 服務](#)
7. [7. 在本機執行服務](#)
8. [8. 部署服務](#)
9. [9. 測試服務](#)
10. [10. 恭喜！](#)
11. [11. 清除所用資源](#)

步驟 1 · 約 0 分鐘

1. 簡介

總覽

在本程式碼研究室中，您將瞭解如何使用 Vertex AI Gemini API 和 Vertex AI 用戶端程式庫，以 Node.js 建立基本的聊天機器人。這個應用程式使用以 [Google Cloud Firestore](#) 為基礎的 [Express](#) 工作階段儲存空間。

課程內容

- 如何使用 htmx、tailwindcss 和 express.js 建構 Cloud Run 服務
 - 如何使用 Vertex AI 用戶端程式庫向 Google API 進行驗證
 - 如何建立聊天機器人，與 Gemini 模型互動
 - 如何部署至 Cloud Run 服務 (不含 Docker 檔案)
 - 如何使用由 Google Cloud Firestore 支援的 Express 工作階段儲存空間
-

步驟 2 · 約 0 分鐘

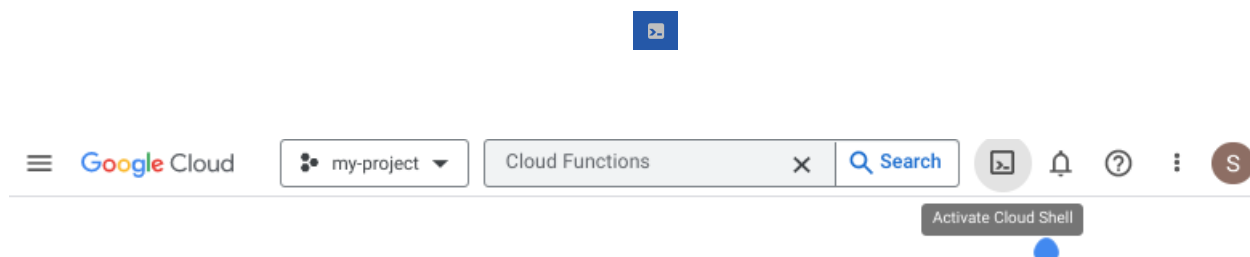
2. 設定和需求

必要條件

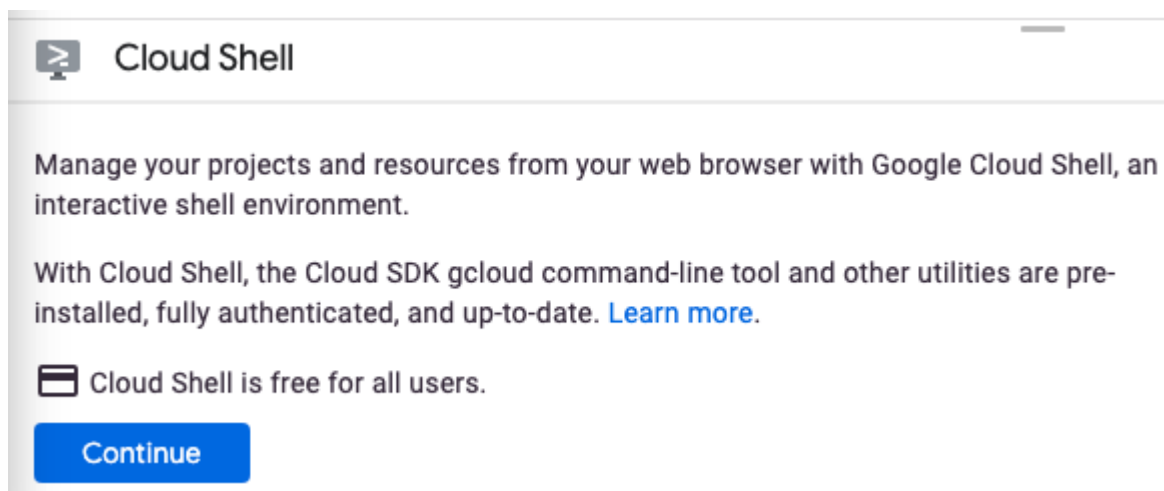
- 您已登入 Cloud Console。
- 您先前已部署 Cloud Run 服務。舉例來說，您可以按照[從原始碼部署網路服務的快速入門導覽課程](#)，開始使用 Cloud Run。

啟用 Cloud Shell

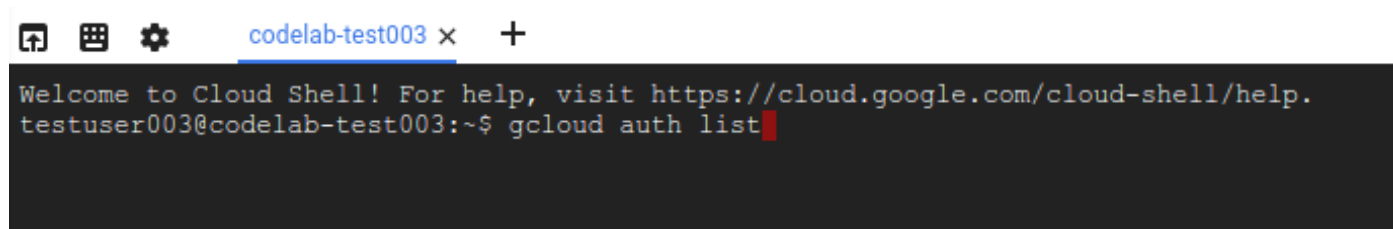
1. 在 Cloud 控制台，點選「啟用 Cloud Shell」圖示



如果您是首次啟動 Cloud Shell，系統會顯示中繼畫面，說明這個指令列環境。如果出現中繼畫面，請按一下「繼續」。



佈建並連至 Cloud Shell 預計只需要幾分鐘。



這部虛擬機器已載入所有必要的開發工具，並提供永久的 5 GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。本程式碼研究室幾乎所有工作都可在瀏覽器上完成。

連至 Cloud Shell 後，您應該會看到驗證已完成，專案也已設為獲派的專案 ID。

2. 在 Cloud Shell 中執行下列指令，確認您已通過驗證：

```
gcloud auth list
```

指令輸出



```
Credentialed Accounts
ACTIVE  ACCOUNT
*       <my_account>@<my_domain.com>

To set the active account, run:
$ gcloud config set account `ACCOUNT`
```

注意： `gcloud` 指令列工具是 Google Cloud 中功能強大的整合式指令列工具，並已預先安裝於 Cloud Shell。你會發現它支援 Tab 鍵自動完成功能。第一次執行指令時，系統可能會提示您進行驗證。詳情請參閱 [gcloud 指令列工具總覽](#)。

3. 在 Cloud Shell 中執行下列指令，確認 `gcloud` 指令知道您的專案：

```
gcloud config list project
```

指令輸出

```
[core]
project = <PROJECT_ID>
```

如未設定，請輸入下列指令手動設定專案：

```
gcloud config set project <PROJECT_ID>
```

指令輸出

```
Updated property [core/project].
```

注意： 如果您在 Cloud Shell 以外的環境完成本教學課程，請按照「[設定應用程式預設憑證](#)」一文的說明操作。

步驟 3 · 約 0 分鐘

3. 啟用 API 並設定環境變數

啟用 API

開始進行本程式碼研究室之前，請先啟用數個 API。本程式碼研究室需要使用下列 API。執行下列指令即可啟用這些 API：

```
gcloud services enable run.googleapis.com \  
  cloudbuild.googleapis.com \  
  aiplatform.googleapis.com \  
  secretmanager.googleapis.com
```

設定環境變數

您可以設定本程式碼實驗室全程都會使用的環境變數。

```
PROJECT_ID=<YOUR_PROJECT_ID>  
REGION=<YOUR_REGION, e.g. us-central1>  
SERVICE=chat-with-gemini  
SERVICE_ACCOUNT="vertex-ai-caller"  
SERVICE_ACCOUNT_ADDRESS=$SERVICE_ACCOUNT@$PROJECT_ID.iam.gserviceaccount.com  
SECRET_ID="SESSION_SECRET"
```

4. 建立及設定 Firebase 專案

1. 在 [Firebase 控制台](#) 中，按一下「新增專案」。
 2. 輸入 <YOUR_PROJECT_ID>，將 Firebase 新增至現有的其中一個 Google Cloud 專案
 3. 如果系統顯示提示，請詳閱並接受 [Firebase 條款](#)。
 4. 按一下「繼續」。
 5. 按一下「確認方案」，確認 Firebase 計費方案。
 6. 您可以選擇是否為這個程式碼研究室啟用 Google Analytics。
 7. 按一下「新增 Firebase」。
 8. 專案建立完成後，請按一下「繼續」。
 9. 在「建構」選單中，按一下「Firestore 資料庫」。
 10. 按一下 [Create database] (建立資料庫)。
 11. 從「位置」下拉式選單中選擇區域，然後按一下「下一步」。
 12. 使用預設的「以正式版模式啟動」，然後按一下「建立」。
-

步驟 5 · 約 0 分鐘

5. 建立服務帳戶

Cloud Run 會使用這個服務帳戶呼叫 Vertex AI Gemini API。這個服務帳戶也會具備 Firestore 的讀取和寫入權限，以及從 Secret Manager 讀取密鑰的權限。

首先，請執行下列指令來建立服務帳戶：

```
gcloud iam service-accounts create $SERVICE_ACCOUNT \
  --display-name="Cloud Run to access Vertex AI APIs"
```

其次，將 Vertex AI 使用者角色授予服務帳戶。

```
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member serviceAccount:$SERVICE_ACCOUNT_ADDRESS \
  --role=roles/aiplatform.user
```

現在，請在 Secret Manager 中建立密鑰。Cloud Run 服務會將這個密鑰當做環境變數存取，並在執行個體啟動時解析。進一步瞭解[密鑰](#)和 [Cloud Run](#)。

```
gcloud secrets create $SECRET_ID --replication-policy="automatic"
printf "keyboard-cat" | gcloud secrets versions add $SECRET_ID --data-file=-
```

並授予服務帳戶 Secret Manager 中 Express 工作階段密鑰的存取權。

```
gcloud secrets add-iam-policy-binding $SECRET_ID \
  --member serviceAccount:$SERVICE_ACCOUNT_ADDRESS \
  --role='roles/secretmanager.secretAccessor'
```

最後，授予服務帳戶 Firestore 的讀取及寫入權限。

```
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member serviceAccount:$SERVICE_ACCOUNT_ADDRESS \
  --role=roles/datastore.user
```

步驟 6 · 約 0 分鐘

6. 建立 Cloud Run 服務

首先，請建立原始碼的目錄，然後 cd 到該目錄。

```
mkdir chat-with-gemini && cd chat-with-gemini
```

接著，請建立 `package.json` 檔案，並加入以下內容：

```
{
  "name": "chat-with-gemini",
  "version": "1.0.0",
  "description": "",
  "main": "app.js",
  "scripts": {
    "start": "node app.js",
    "nodemon": "nodemon app.js",
    "cssdev": "npx tailwindcss -i ./input.css -o ./public/output.css --watch",
    "tailwind": "npx tailwindcss -i ./input.css -o ./public/output.css",
    "dev": "npm run tailwind && npm run nodemon"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "@google-cloud/connect-firestore": "^3.0.0",
    "@google-cloud/firestore": "^7.5.0",
    "@google-cloud/vertexai": "^0.4.0",
    "axios": "^1.6.8",
    "express": "^4.18.2",
    "express-session": "^1.18.0",
    "express-ws": "^5.0.2",
    "htmx.org": "^1.9.10"
  },
  "devDependencies": {
    "nodemon": "^3.1.0",
    "tailwindcss": "^3.4.1"
  }
}
```

接著，請使用下列內容建立 `app.js` 來源檔案。這個檔案包含服務的進入點，以及應用程式的主要邏輯。

```

const express = require("express");
const app = express();
app.use(express.urlencoded({ extended: true }));
app.use(express.json());
const path = require("path");

const fs = require("fs");
const util = require("util");
const { spinnerSvg } = require("./spinnerSvg.js");

// cloud run retrieves secret at instance startup time
const secret = process.env.SESSION_SECRET;

const { Firestore } = require("@google-cloud/firestore");
const { FirestoreStore } = require("@google-cloud/connect-firestore");
var session = require("express-session");
app.set("trust proxy", 1); // trust first proxy
app.use(
  session({
    store: new FirestoreStore({
      dataset: new Firestore(),
      kind: "express-sessions"
    }),
    secret: secret,
    /* set secure to false for local dev session history testing */
    /* see more at https://expressjs.com/en/resources/middleware/session.html */
    cookie: { secure: true },
    resave: false,
    saveUninitialized: true
  })
);

const expressWs = require("express-ws")(app);

app.use(express.static("public"));

// Vertex AI Section
const { VertexAI } = require("@google-cloud/vertexai");

// instance of Vertex model
let generativeModel;

// on startup
const port = parseInt(process.env.PORT) || 8080;
app.listen(port, async () => {
  console.log(`demo1: listening on port ${port}`);

  // get project and location from metadata service

```

```

const metadataService = require("./metadataService.js");

const project = await metadataService.getProjectId();
const location = await metadataService.getRegion();

// Vertex client library instance
const vertex_ai = new VertexAI({
  project: project,
  location: location
});

// Instantiate models
generativeModel = vertex_ai.getGenerativeModel({
  model: "gemini-1.0-pro-001"
});
});

app.ws("/sendMessage", async function (ws, req) {
  if (!req.session.chathistory || req.session.chathistory.length == 0) {
    req.session.chathistory = [];
  }

  let chatWithModel = generativeModel.startChat({
    history: req.session.chathistory
  });

  ws.on("message", async function (message) {

    console.log("req.sessionID: ", req.sessionID);
    // get session id

    let questionToAsk = JSON.parse(message).message;
    console.log("WebSocket message: " + questionToAsk);

    ws.send(`<div hx-swap-oob="beforeend:#toupdate"><div
      id="questionToAsk"
      class="text-black m-2 text-right border p-2 rounded-lg ml-
24">

        ${questionToAsk}
      </div></div>`);

    // to simulate a natural pause in conversation
    await sleep(500);

    // get timestamp for div to replace
    const now = "fromGemini" + Date.now();

    ws.send(`<div hx-swap-oob="beforeend:#toupdate"><div

```

```

                id=${now}
                class=" text-blue-400 m-2 text-left border p-2 rounded-lg mr-
24">

                ${spinnerSvg}
            </div></div>`);

const results = await chatWithModel.sendMessage(questionToAsk);
const answer =
    results.response.candidates[0].content.parts[0].text;

ws.send(`<div
                id=${now}
                hx-swap-oob="true"
                hx-swap="outerHTML"
                class="text-blue-400 m-2 text-left border p-2 rounded-lg mr-
24">

                ${answer}
            </div>`);

        // save to current chat history
let userHistory = {
    role: "user",
    parts: [{ text: questionToAsk }]
};
let modelHistory = {
    role: "model",
    parts: [{ text: answer }]
};

req.session.chathistory.push(userHistory);
req.session.chathistory.push(modelHistory);

// console.log(
//     "newly saved chat history: ",
//     util.inspect(req.session.chathistory, {
//         showHidden: false,
//         depth: null,
//         colors: true
//     })
// );
req.session.save();
});

ws.on("close", () => {
    console.log("WebSocket was closed");
});
});

```

```
function sleep(ms) {
  return new Promise((resolve) => {
    setTimeout(resolve, ms);
  });
}

// gracefully close the web sockets
process.on("SIGTERM", () => {
  server.close();
});
```

建立 tailwindCSS 的 `tailwind.config.js` 檔案。

```
/** @type {import('tailwindcss').Config} */
module.exports = {
  content: ["./**/*.html", "js"],
  theme: {
    extend: {}
  },
  plugins: []
};
```

建立 `metadataService.js` 檔案，取得已部署 Cloud Run 服務的專案 ID 和區域。這些值會用於例項化 Vertex AI 用戶端程式庫的執行個體。

注意：如要在本機執行應用程式，您必須在下列第 1 和第 2 行設定專案 ID 和區域：

```

const your_project_id = "YOUR_PROJECT_ID";
const your_region = "YOUR_REGION";

const axios = require("axios");

module.exports = {
  getProjectId: async () => {
    let project = "";
    try {
      // Fetch the token to make a GCF to GCF call
      const response = await axios.get(
        "http://metadata.google.internal/computeMetadata/v1/project/project-
id",
        {
          headers: {
            "Metadata-Flavor": "Google"
          }
        }
      );

      if (response.data == "") {
        // running locally on Cloud Shell
        project = your_project_id;
      } else {
        // running on Clodu Run. Use project id from metadata service
        project = response.data;
      }
    } catch (ex) {
      // running locally on local terminal
      project = your_project_id;
    }

    return project;
  },

  getRegion: async () => {
    let region = "";
    try {
      // Fetch the token to make a GCF to GCF call
      const response = await axios.get(
        "http://metadata.google.internal/computeMetadata/v1/instance/region",
        {
          headers: {
            "Metadata-Flavor": "Google"
          }
        }
      );
    }
  }
};

```

```

    if (response.data == "") {
      // running locally on Cloud Shell
      region = your_region;
    } else {
      // running on Clodu Run. Use region from metadata service
      let regionFull = response.data;
      const index = regionFull.lastIndexOf("/");
      region = regionFull.substring(index + 1);
    }
  } catch (ex) {
    // running locally on local terminal
    region = your_region;
  }
  return region;
}
};

```

建立名為 `spinnerSvg.js` 的檔案

```

module.exports.spinnerSvg = `

```

最後，為 `tailwindCSS` 建立 `input.css` 檔案。

```

@tailwind base;
@tailwind components;
@tailwind utilities;

```

現在，請建立新的 `public` 目錄。

```
mkdir public  
cd public
```

並在該公開目錄中，為前端建立 `index.html` 檔案，該檔案將使用 `htmx`。


```

<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta
      name="viewport"
      content="width=device-width, initial-scale=1.0"
    />
    <script
      src="https://unpkg.com/htmx.org@1.9.10"
      integrity="sha384-
D1Kt99CQMDuVetoL1lrYwg5t+9QdHe7NLX/SoJYkXDFfx37iInKRy5xLSi8nO7UC"
      crossorigin="anonymous"
    ></script>

    <link href="./output.css" rel="stylesheet" />
    <script src="https://unpkg.com/htmx.org/dist/ext/ws.js"></script>

    <title>Demo 1</title>
  </head>
  <body>
    <div id="herewego" text-center>
      <!-- <div id="replaceme2" hx-swap-oob="true">Hello world</div> -->
      <div
        class="container mx-auto mt-8 text-center max-w-screen-lg"
      >
        <div
          class="overflow-y-scroll bg-white p-2 border h-[500px] space-y-4
rounded-lg m-auto"
        >
          <div id="toupdate"></div>
        </div>
        <form
          hx-trigger="submit, keyup[keyCode==13] from:body"
          hx-ext="ws"
          ws-connect="/sendMessage"
          ws-send=""
          hx-on="htmx:wsAfterSend: document.getElementById('message').value
= ''"
        >
          <div class="mb-6 mt-6 flex gap-4">
            <textarea
              rows="2"
              type="text"
              id="message"
              name="message"
              class="block grow rounded-lg border p-6 resize-none"
              required

```

```
        >
Is C# a programming language or a musical note?</textarea>
        >
        <button
            type="submit"
            class="bg-blue-500 text-white px-4 py-2 rounded-lg text-
center text-sm font-medium"
        >
            Send
        </button>
    </div>
</form>
</div>
</div>
</body>
</html>
```

步驟 7 · 約 0 分鐘

7. 在本機執行服務

首先，請確認您位於程式碼研究室的根目錄 `chat-with-gemini`。

```
cd .. && pwd
```

接著，執行下列指令來安裝依附元件：

```
npm install
```

在本機執行時使用 ADC

如果您在 Cloud Shell 中執行，表示您已在 Google Compute Engine 虛擬機器上執行。應用程式預設憑證會自動使用與這個虛擬機器相關聯的憑證 (如執行 `gcloud auth list` 所示)，因此不必使用 `gcloud auth application-default login` 指令。您可以跳至「建立本機工作階段密鑰」一節

不過，如果您是在本機終端機上執行 (即不在 Cloud Shell 中)，則需要使用應用程式預設憑證向 Google API 進行驗證。您可以選擇 1) 使用憑證登入 (前提是您同時具備 Vertex AI 使用者和資料儲存庫使用者角色)，或 2) 模擬本程式碼實驗室中使用的服務帳戶登入。

選項 1) 使用 ADC 的憑證

如要使用憑證，請先執行 `gcloud auth list`，確認您在 `gcloud` 中的驗證方式。接著，您可能需要授予身分「Vertex AI 使用者」角色。如果您的身分具備「擁有者」角色，您就已擁有這個 Vertex AI 使用者角色。如果沒有，可以執行這項指令，將 Vertex AI 使用者角色和 Datastore 使用者角色授予您的身分。

```
USER=<YOUR_PRINCIPAL_EMAIL>

gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member user:$USER \
  --role=roles/aiplatform.user

gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member user:$USER \
  --role=roles/datastore.user
```

然後執行下列指令

```
gcloud auth application-default login
```

選項 2) 模擬 ADC 的服務帳戶

如要使用本程式碼研究室建立的服務帳戶，您的使用者帳戶必須具備[服務帳戶權杖建立者角色](#)。您可以執行下列指令來取得這個角色：

```
gcloud projects add-iam-policy-binding $PROJECT_ID \  
  --member user:$USER \  
  --role=roles/iam.serviceAccountTokenCreator
```

接著，執行下列指令，搭配服務帳戶使用 ADC

```
gcloud auth application-default login --impersonate-service-  
account=$SERVICE_ACCOUNT_ADDRESS
```

建立本機工作階段密鑰

現在，請為本機開發作業建立本機工作階段密鑰。

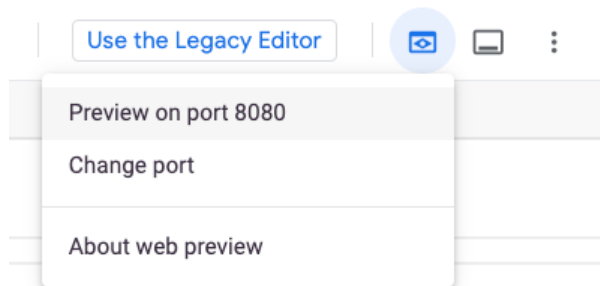
```
export SESSION_SECRET=local-secret
```

在本機執行應用程式

最後，您可以執行下列指令碼來啟動應用程式。這個指令碼也會從 tailwindCSS 產生 output.css 檔案。

```
npm run dev
```

開啟「網頁預覽」按鈕，然後選取「預覽通訊埠 8080」，即可預覽網站



步驟 8 · 約 0 分鐘

8. 部署服務

首先，請執行下列指令啟動部署作業，並指定要使用的服務帳戶。如果未指定服務帳戶，即會使用預設的運算服務帳戶。

```
gcloud run deploy $SERVICE \  
  --service-account $SERVICE_ACCOUNT_ADDRESS \  
  --source . \  
  --region $REGION \  
  --allow-unauthenticated \  
  --set-secrets="SESSION_SECRET=$(echo $SECRET_ID):1"
```

如果系統提示「Deploying from source requires an Artifact Registry Docker repository to store built containers. 系統會在 [us-central1] 區域建立名為 [cloud-run-source-deploy] 的存放區。」，請按下「y」接受並繼續。

步驟 9 · 約 0 分鐘

9. 測試服務

部署完成後，請在網路瀏覽器中開啟服務網址。然後問問 Gemini，例如「我會彈吉他，也是軟體工程師。看到「C#」時，我應該將其視為程式設計語言還是音符？我該選擇哪一個？」

步驟 10 · 約 0 分鐘

10. 恭喜！

恭喜您完成本程式碼研究室！

建議您參閱 [Cloud Run](#) 和 [Vertex AI Gemini API](#) 說明文件。

涵蓋內容

- 如何使用 htmx、tailwindcss 和 express.js 建構 Cloud Run 服務
 - 如何使用 Vertex AI 用戶端程式庫向 Google API 進行驗證
 - 如何建立聊天機器人，與 Gemini 模型互動
 - 如何部署至 Cloud Run 服務 (不含 Docker 檔案)
 - 如何使用由 Google Cloud Firestore 支援的 Express 工作階段儲存空間
-

11. 清除所用資源

為避免產生意外費用 (例如，Cloud Run 服務意外叫用次數超過[免費層級的每月 Cloud Run 叫用次數配額](#))，您可以刪除 Cloud Run，或刪除您在步驟 2 中建立的專案。

如要刪除 Cloud Run 服務，請前往 Cloud Run Cloud 控制台 (<https://console.cloud.google.com/run>)，然後刪除 `chat-with-gemini` 服務。您也可以刪除 `vertex-ai-caller` 服務帳戶或撤銷 Vertex AI 使用者角色，避免無意間呼叫 Gemini。

如要刪除整個專案，請前往 <https://console.cloud.google.com/cloud-resource-manager>，選取您在步驟 2 中建立的專案，然後選擇「刪除」。刪除專案後，您必須在 Cloud SDK 中變更專案。如要查看所有可用專案的清單，請執行 `gcloud projects list`。
